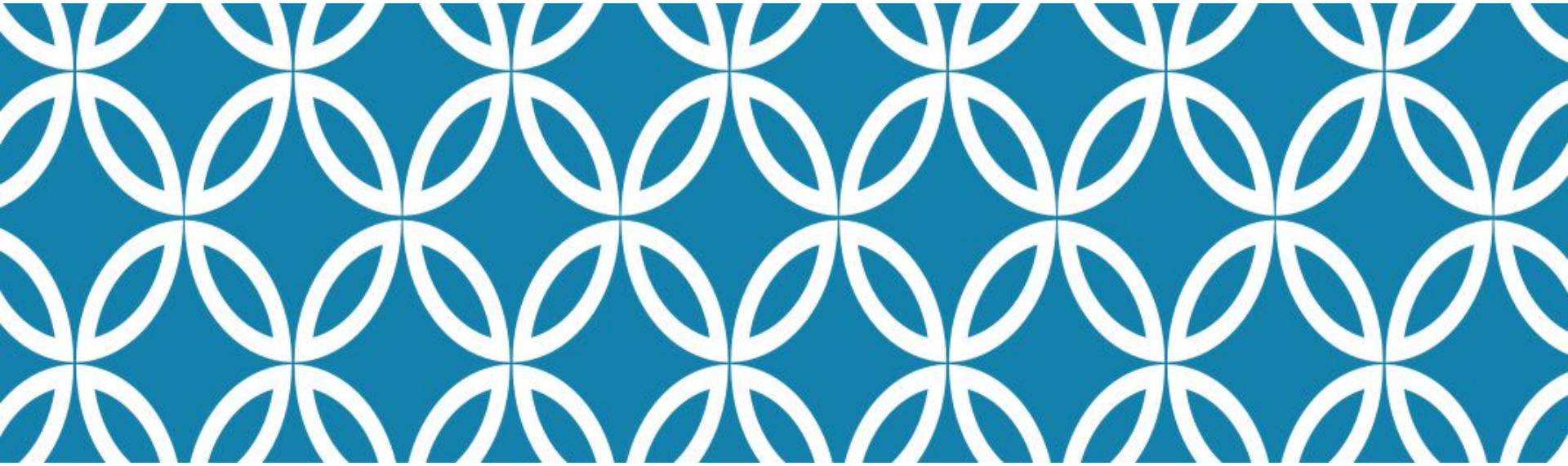
Advance Database



TD03
Database load balancing

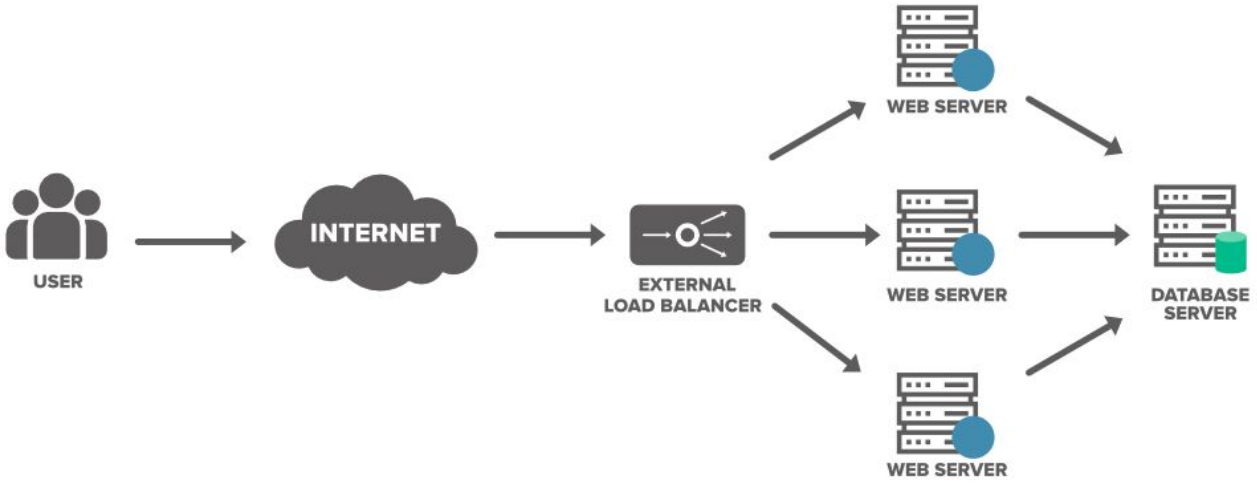
Prepared by Mr. Nop Phearum

Outline

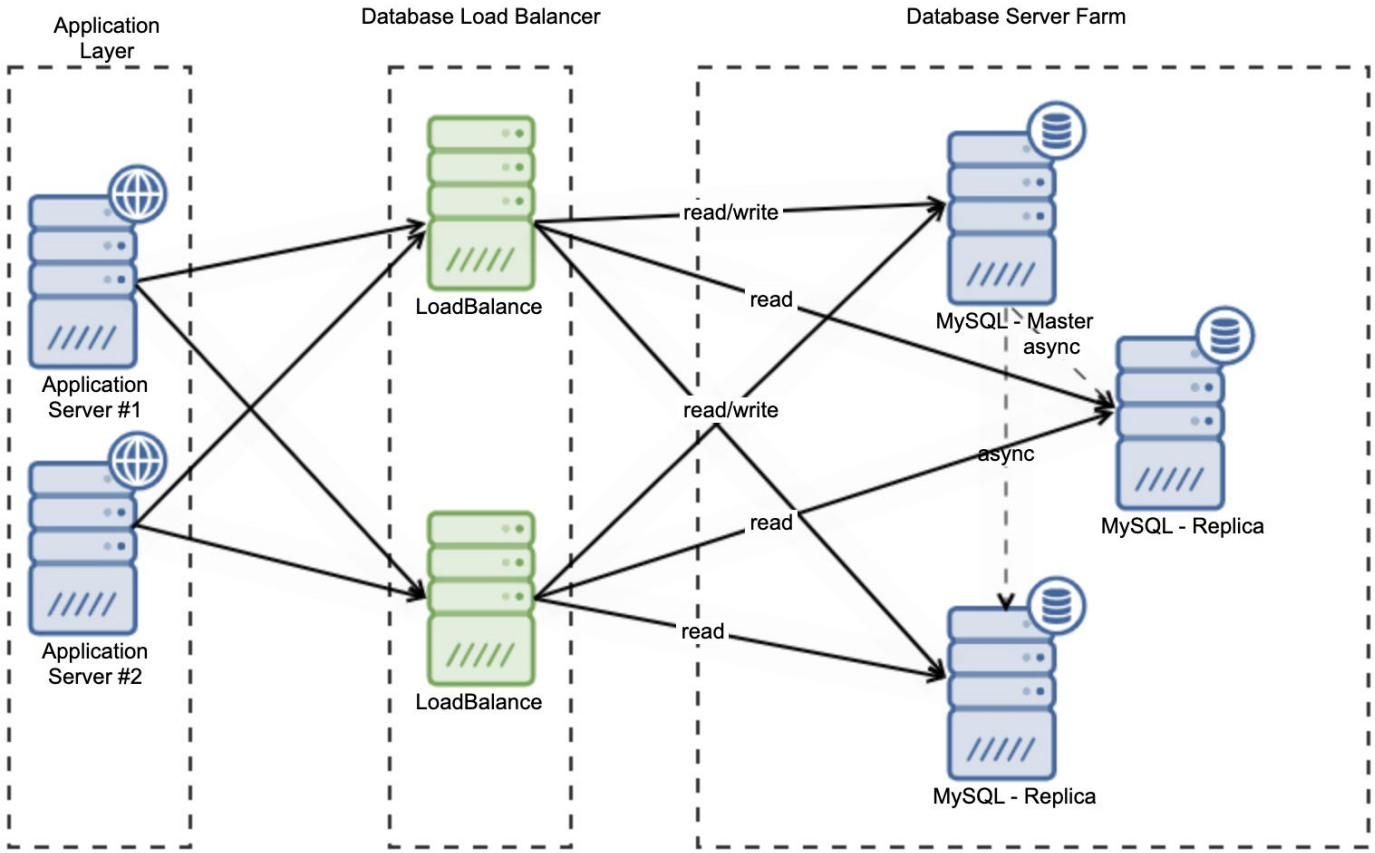
- ➔ Methods of Database Load Balancing
- ➔ Implementing Read/Write Splitting (TOPIC)
- ➔ Load Balancing Database Clusters (TOPIC)
- ➔ Database Sharding (TOPIC)

Methods of Database Load Balancing

- Different techniques for balancing load across database servers, like round-robin DNS, hardware load balancers, application-level balancing, database replication, client-side load balancing.
- Compare pros and cons of each approach.



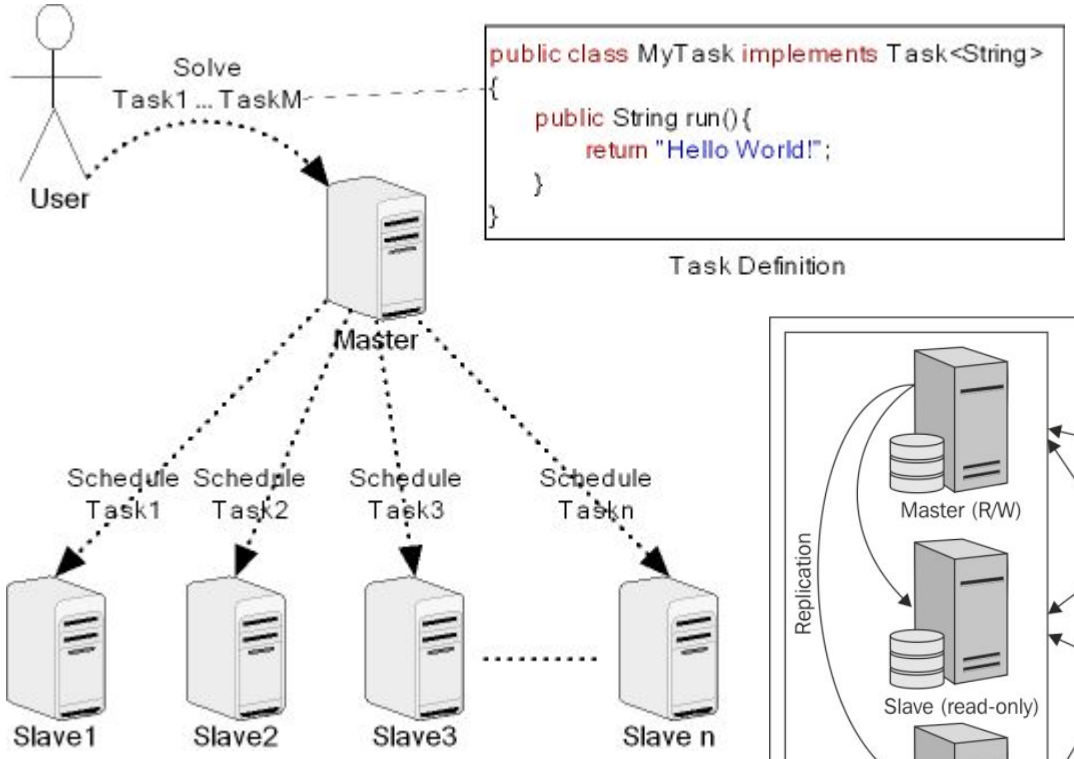
Methods of Database Load Balancing



Implementing Read/Write Splitting

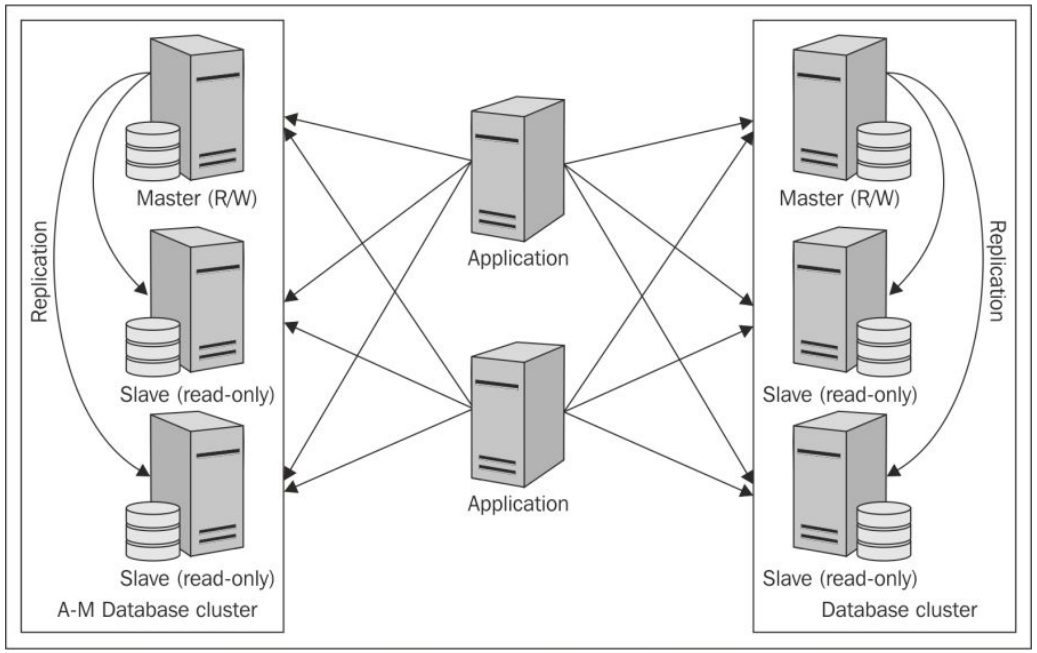
- The concept of splitting reads and writes across different database servers to improve performance and scalability. How to set up master-slave replication, implement query routing logic to send reads to slaves.
- Cover challenges like latency of data propagation.

Implementing Read/Write Splitting



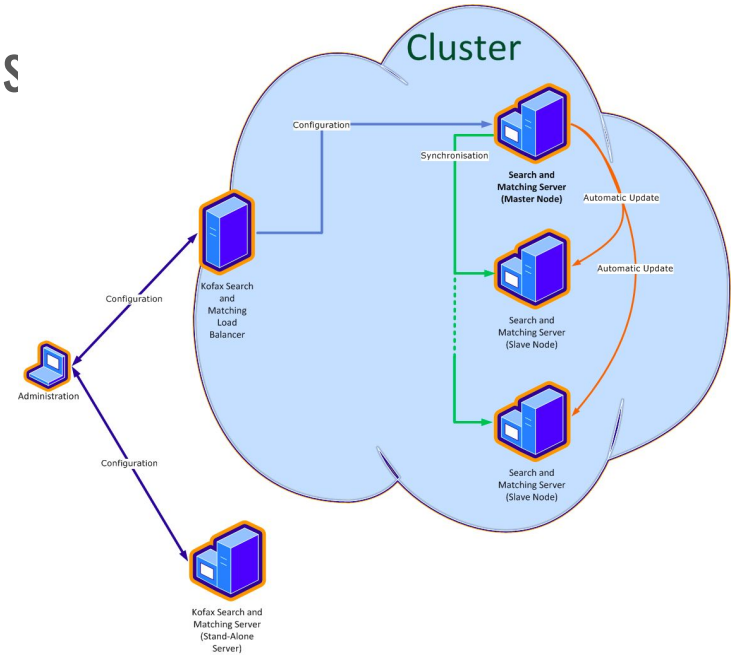

```
public class MyTask implements Task<String>
{
    public String run(){
        return "Hello World!";
    }
}
```

Task Definition

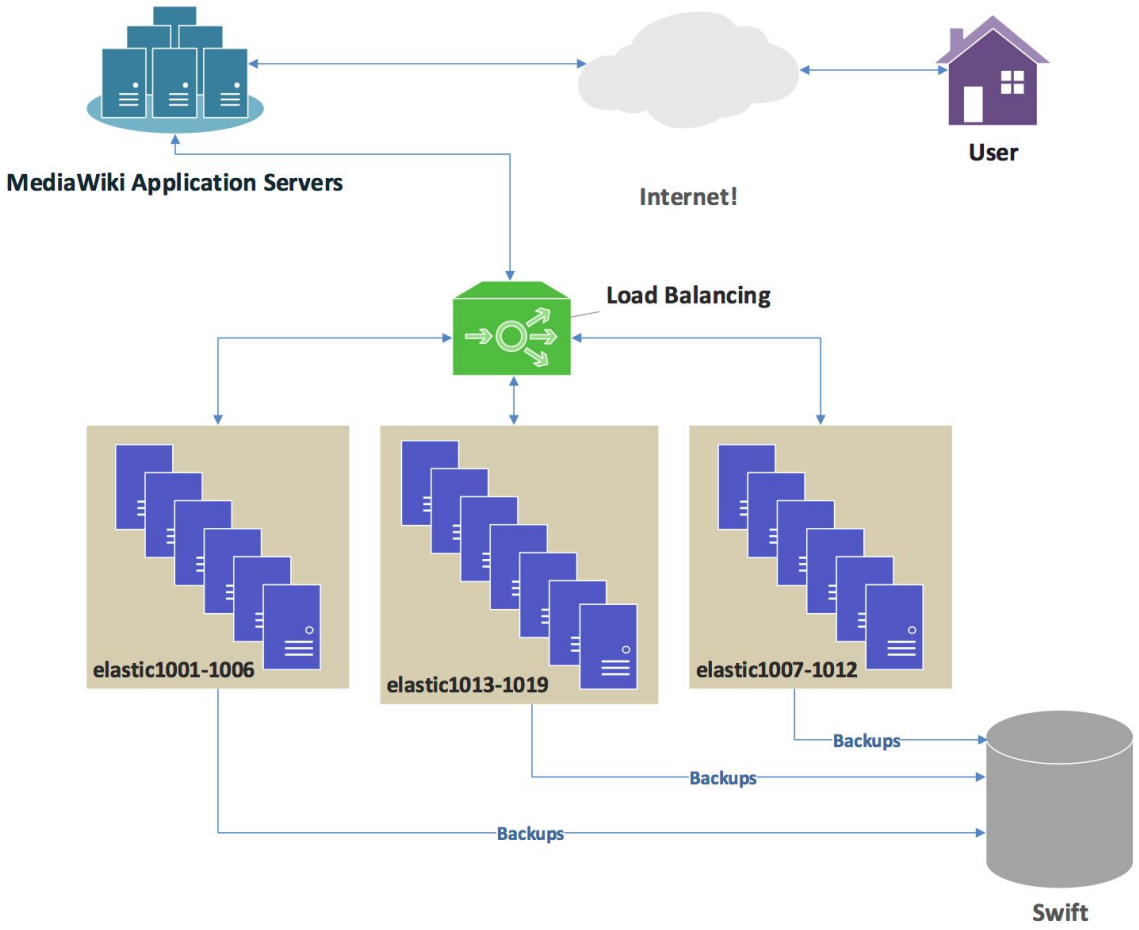


Load Balancing Database Clusters

- Load balancing a clustered database like MySQL Cluster or Oracle RAC across multiple nodes.
- The built-in transparency and load balancing capabilities.
- Analyze configuring, monitoring and managing a load-balanced database clus



Load Balancing Database Clusters



Load Balancing Database Clusters

Database clustering is a technique for improving database performance, availability, and scalability by distributing database operations across multiple interconnected servers. Key characteristics include:

- **High Availability** : Ensures continuous database operation even if one node fails
- **Horizontal Scaling** : Allows adding more nodes to increase overall system capacity
- **Shared-Nothing Architecture** : Each node operates independently with its own memory and storage

Database Sharding

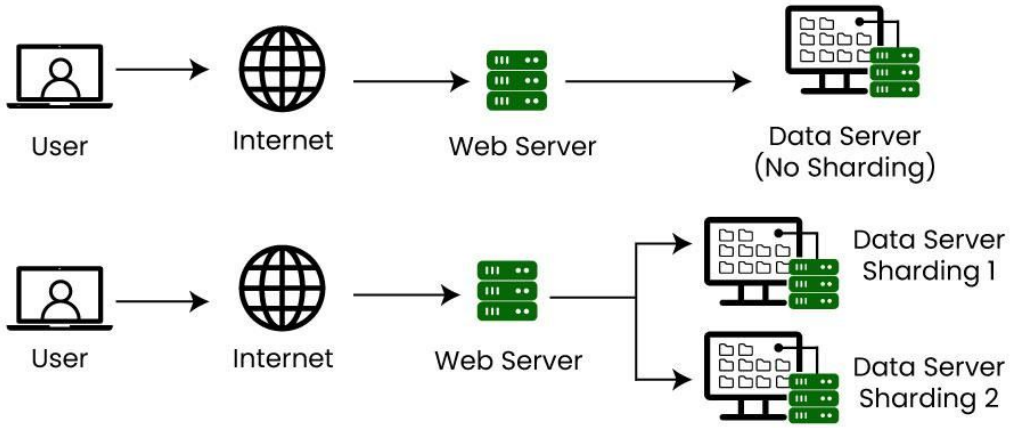
Sharding is a database partitioning technique that splits large databases into smaller, more manageable pieces called shards. Each shard contains a subset of the total data, distributed across multiple servers.

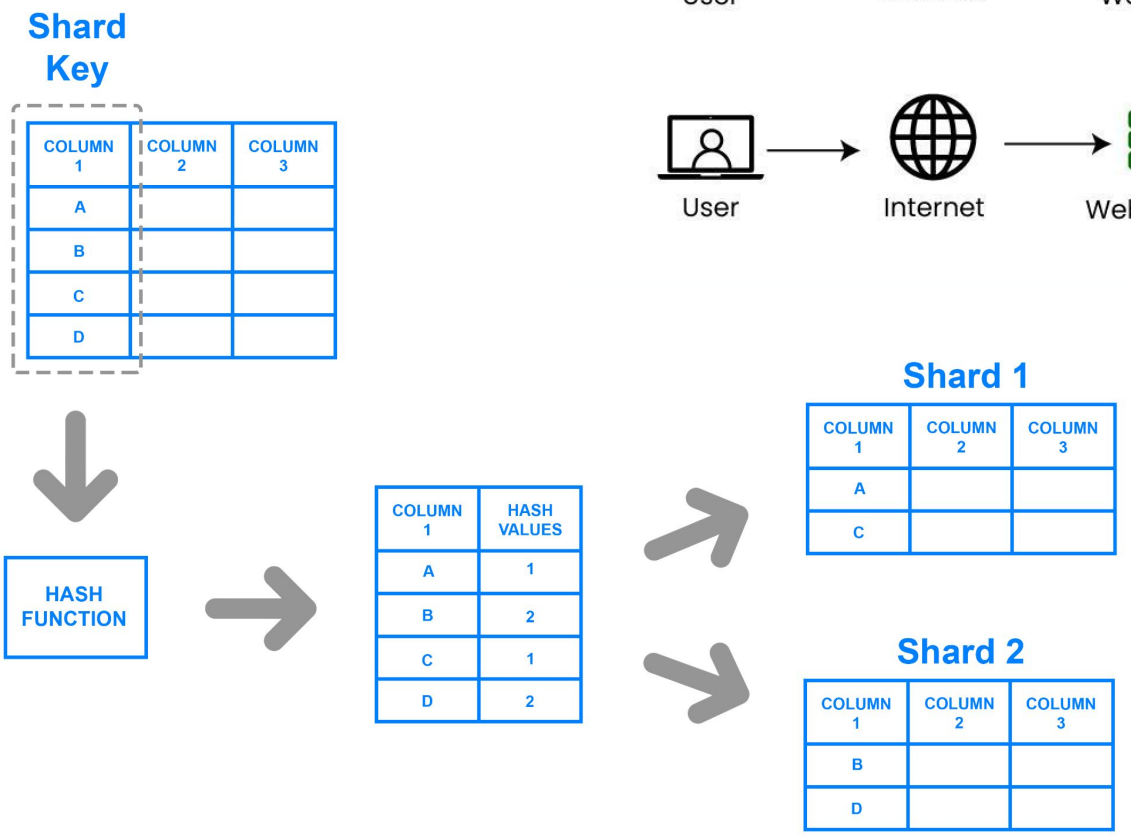
Sharding Strategies:

- Horizontal Sharding (Row-based)
  - Distributes rows of a table across multiple database instances
  - Suitable for large tables with similar row structures
  - Example: Splitting user data based on user ID ranges
- Vertical Sharding (Column-based)
  - Splits tables by columns
  - Separates frequently accessed from rarely accessed columns
  - Useful for optimizing specific query patterns

Database Sharding

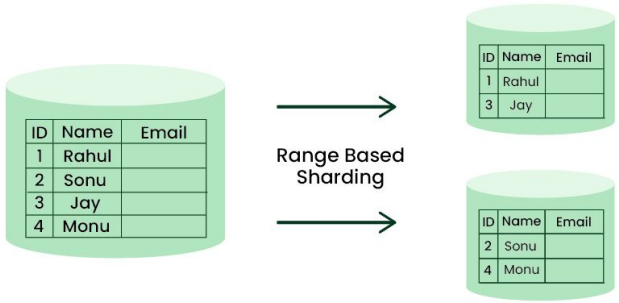
Sharding Strategies:



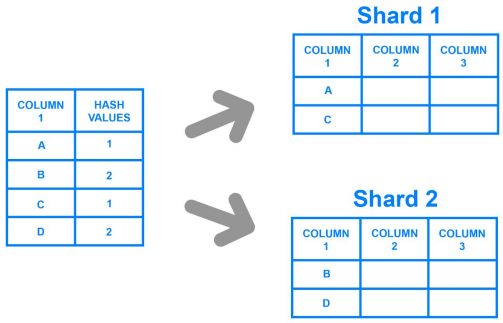


Database Sharding

Sharding Techniques:



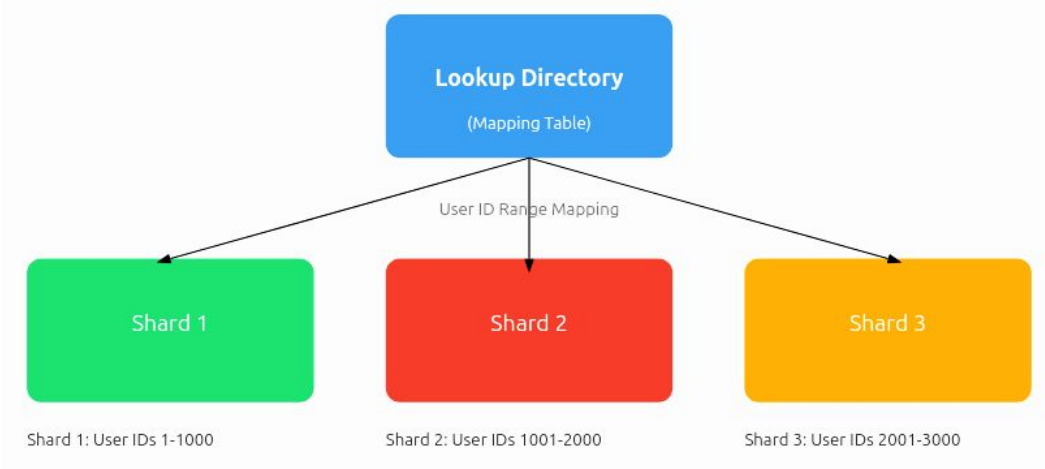
- Range-based Sharding
  - Divide data based on value ranges
  - Simple to implement
  - Can lead to uneven data distribution



- Hash-based Sharding
  - Use hash function to determine shard location
  - More uniform data distribution
  - Consistent hashing helps minimize data movement during scaling

Database Sharding

Sharding Techniques:



- Directory-based Sharding
  - Maintain a lookup table to map data to specific shards
  - Most flexible approach
  - Allows complex sharding logic

Database Sharding

Tools and Frameworks:

- Vitess: Sharding solution for MySQL
- Citus: Distributed PostgreSQL extension
- Apache ShardingSphere: Database middleware for sharding
- MongoDB Sharding: Built-in sharding capabilities

TOPIC

ABOUT GROUP

1 big group = 3 topics

1 big group / [9,10] small groups

1 small group / 3 members

GROUP 1

1. Master-Slave Replication: Set up master-slave replication where reads are handled by multiple slave servers while writes go to the master. Requires configuring replication in PostgreSQL/MySQL and implementing application-side read/write splitting.

2. PostgreSQL Streaming Replication: Use PostgreSQL's native streaming replication to have a primary server that handles writes and multiple secondaries that handle reads. Transactions are streamed to secondary servers in real-time. Needs configuring standby servers in recovery mode.

3. MySQL InnoDB Cluster: Deploy an InnoDB cluster between multiple MySQL instances for auto-scaling, auto failover and load balancing. Handles node provisioning and distributed recovery automatically. Needs configuring the MySQL router and individual instances.

GROUP 2

4. pgpool-II connection pooler: Use pgpool-II between the application and PostgreSQL servers for connection pooling, replication, load balancing, limiting connections etc. Needs installing pgpool and tweaking pgpool.conf configuration file.

5. ProxySQL Load Balancer: Put ProxySQL in front of MySQL database servers which can aggregate query results and route transactions while balancing reading and writing. Requires installing and configuring ProxySQL instance.

6. Hardware Load Balancer: Use a dedicated hardware load balancer like F5 or HAProxy to distribute load across multiple PostgreSQL or MySQL instances. Needs device configuration of load balancing method like round robin etc.

GROUP 3

7. Database Sharding: Challenges in Sharding

- ❏ Complex Query Routing: Determining which shard to query
- ❏ Data Consistency: Maintaining ACID properties
- ❏ Cross-Shard Joins: Difficult and performance-intensive
- ❏ Resharding: Redistributing data when adding or removing nodes

Best Practices

- ❏ Choose sharding key carefully
- ❏ Design for horizontal scalability from the start
- ❏ Implement robust monitoring and management tools
- ❏ Consider eventual consistency models
- ❏ Plan for data migration and rebalancing